



Computer Vision and Image Processing

Lecture

Noise Models

Noise models provide a **formal, mathematical representation of how unwanted disturbances affect images**. Without noise models, it would not be possible to **analyze, simulate, remove, or design systems robust to noise** in a systematic way.

1. Gaussian Noise (Additive Noise)
2. Salt & Pepper Noise (Impulse Noise)
3. Speckle Noise (Multiplicative Noise)

Gaussian Noise (Additive Noise)

Gaussian noise is a type of **statistical noise** where the noise values follow a **normal (Gaussian) probability distribution**.

It usually occurs due to **sensor noise, poor illumination, or electronic interference**.

Mathematical Model

$$g(x, y) = f(x, y) + n(x, y)$$

- $f(x, y)$ → original image
- $n(x, y)$ → Gaussian noise with mean μ and variance σ^2
- $g(x, y)$ → noisy image

Gaussian Noise (Additive Noise)

Matrix Example

Original Image Matrix

$$F = \begin{bmatrix} 100 & 102 & 104 \\ 98 & 100 & 102 \\ 96 & 98 & 100 \end{bmatrix}$$

Gaussian Noise Matrix

$$N = \begin{bmatrix} 2 & -1 & 0 \\ -2 & 1 & 3 \\ 0 & -1 & 2 \end{bmatrix}$$

Gaussian Noise (Additive Noise)

Add corresponding elements:

$$G = F + N$$

$$G = \begin{bmatrix} 100 + 2 & 102 - 1 & 104 + 0 \\ 98 - 2 & 100 + 1 & 102 + 3 \\ 96 + 0 & 98 - 1 & 100 + 2 \end{bmatrix}$$

Resulting Noisy Image

$$G = \begin{bmatrix} 102 & 101 & 104 \\ 96 & 101 & 105 \\ 96 & 97 & 102 \end{bmatrix}$$

Salt & Pepper Noise (Impulse Noise)

Salt and Pepper noise is an **impulse noise** where pixels are randomly replaced by **minimum (0)** or **maximum (255)** intensity values.

- **Salt noise** → bright white pixels
- **Pepper noise** → dark black pixels

Mathematical Representation

$$g(x, y) = \begin{cases} 0 & \text{with probability } p \\ 255 & \text{with probability } q \\ f(x, y) & \text{otherwise} \end{cases}$$

Salt & Pepper Noise (Impulse Noise)

- **Step-by-Step Explanation**

1. Random pixels are selected.
2. Selected pixels are replaced with:
 - **255 (salt)** or
 - **0 (pepper)**
3. Remaining pixels stay unchanged.

Original Image

$$F = \begin{bmatrix} 120 & 122 & 124 \\ 118 & 120 & 122 \\ 116 & 118 & 120 \end{bmatrix}$$

Injected Salt & Pepper Noise

$$G = \begin{bmatrix} 120 & 255 & 124 \\ 0 & 120 & 122 \\ 116 & 118 & 0 \end{bmatrix}$$

Speckle Noise (Multiplicative Noise)

Speckle noise is a **multiplicative noise** commonly found in **radar, ultrasound, and medical imaging**.

Mathematical Model

$$g(x, y) = f(x, y) + f(x, y) \times n(x, y)$$

or

$$g(x, y) = f(x, y)(1 + n(x, y))$$

Speckle Noise (Multiplicative Noise)

Original Image

$$F = \begin{bmatrix} 50 & 52 & 54 \\ 48 & 50 & 52 \\ 46 & 48 & 50 \end{bmatrix}$$

Speckle Noise Matrix

$$N = \begin{bmatrix} 0.1 & -0.05 & 0 \\ 0.2 & -0.1 & 0.05 \\ 0 & 0.1 & -0.05 \end{bmatrix}$$

Speckle Noise (Multiplicative Noise)

Final Speckled Image Matrix

$$G = \begin{bmatrix} 55 & 49.4 & 54 \\ 57.6 & 45 & 54.6 \\ 46 & 52.8 & 47.5 \end{bmatrix}$$

Noise Removal Filters

Noise Removal Filters are techniques in image processing used to **reduce or remove unwanted disturbances (noise) from images**. Noise can be introduced during image acquisition, transmission, or storage, and it degrades image quality by altering pixel values randomly or systematically. The goal of these filters is to restore the image as close as possible to its original state while preserving important details like edges and textures.

1. Mean Filter (Linear Smoothing Filter)
2. Adaptive Filter
3. Order-Statistics Filter (Median Filter)
4. Inverse Filtering
5. Wiener Filter

Mean Filter (Linear Smoothing Filter)

Mean Filter Reduces random noise by averaging the pixel values in a neighborhood.

Formula:

$$g(i, j) = \frac{1}{m \times n} \sum_{k=-a}^a \sum_{l=-b}^b f(i + k, j + l)$$

Where $m \times n$ is the size of the filter kernel (e.g., 3×3).

Mean Filter (Linear Smoothing Filter)

Mean Filter Reduces random noise by averaging the pixel values in a neighborhood.

Formula:

$$g(i, j) = \frac{1}{m \times n} \sum_{k=-a}^a \sum_{l=-b}^b f(i + k, j + l)$$

Where $m \times n$ is the size of the filter kernel (e.g., 3×3).

Mean Filter (Linear Smoothing Filter)

Noisy image patch (3×3):

$$f = \begin{bmatrix} 52 & 55 & 61 \\ 68 & 70 & 65 \\ 61 & 59 & 60 \end{bmatrix}$$

Step 1: Apply 3×3 mean filter:

$$\text{Mean} = \frac{52 + 55 + 61 + 68 + 70 + 65 + 61 + 59 + 60}{9} = \frac{551}{9} \approx 61.22$$

Step 2: Replace central pixel (70) with 61.22.

Resulting patch:

$$g = \begin{bmatrix} 52 & 55 & 61 \\ 68 & 61 & 65 \\ 61 & 59 & 60 \end{bmatrix}$$

Adaptive Filter

Adaptive Filter adjusts the filter according to local variance. Good for non-uniform noise.

Formula (for pixel (i, j)):

$$g(i, j) = f(i, j) - \frac{\sigma_n^2}{\sigma_L^2} [f(i, j) - \mu_L]$$

Where:

- σ_n^2 = noise variance
- σ_L^2 = local variance of neighborhood
- μ_L = local mean

Adaptive Filter

Noisy patch:

$$f = \begin{bmatrix} 100 & 102 & 103 \\ 99 & 105 & 104 \\ 101 & 100 & 102 \end{bmatrix}$$

Assume $\sigma_n^2 = 4$.

Adaptive Filter

Step 1: Compute local mean $\mu_L = \frac{100+102+103+99+105+104+101+100+102}{9} = \frac{916}{9} \approx 101.78$

Step 2: Compute local variance $\sigma_L^2 = \frac{\sum(f(i,j) - \mu_L)^2}{9} \approx 5.73$

Step 3: Apply adaptive filter to central pixel (105):

$$g(2, 2) = 105 - \frac{4}{5.73}(105 - 101.78) \approx 105 - 2.28 \approx 102.72$$

Resulting central pixel: 102.72 (smoothed according to local noise).

Order-Statistics Filter (Median Filter)

Order-Statistics Filter (Median Filter) Removes **salt & pepper noise**. Uses median instead of mean.

Step:

1. Sort the pixel values in the neighborhood.
2. Replace the center pixel with the median.

Order-Statistics Filter (Median Filter)

Noisy patch:

$$f = \begin{bmatrix} 10 & 255 & 10 \\ 10 & 100 & 10 \\ 10 & 10 & 10 \end{bmatrix}$$

Step 1: Flatten neighborhood: [10, 255, 10, 10, 100, 10, 10, 10, 10]

Step 2: Sort: [10, 10, 10, 10, 10, 10, 10, 100, 255]

Step 3: Median = 10 → replace center pixel (100) with 10.

Resulting patch:

$$g = \begin{bmatrix} 10 & 255 & 10 \\ 10 & 10 & 10 \\ 10 & 10 & 10 \end{bmatrix}$$

Inverse Filtering

Inverse Filtering Removes blur caused by a linear system $H(u, v)$. Works in frequency domain.

Formula:

$$\hat{F}(u, v) = \frac{G(u, v)}{H(u, v)}$$

Where:

- $G(u, v)$ = degraded image in frequency domain
- $H(u, v)$ = degradation function (e.g., motion blur)
- $\hat{F}(u, v)$ = estimated original image

Inverse Filtering

Image patch (1D for simplicity):

$$f = [10, 20, 30]$$

Degradation kernel $h = [1, 1, 1]$ (simple averaging blur)

- Convolution: $g = f * h = [10, 30, 60, 50, 30]$
- To restore f , divide in frequency domain: $F = G/H \rightarrow$ approximate inverse filter

Step:

$$F \approx [10, 20, 30]$$

Note: Inverse filter amplifies noise if $H(u, v)$ has small values.

Wiener Filter

Wiener Filter Minimizes **mean square error**. Works for both noise and blur.

Formula (frequency domain):

$$\hat{F}(u, v) = \frac{H^*(u, v)}{|H(u, v)|^2 + S_n(u, v)/S_f(u, v)} G(u, v)$$

Where:

- $H(u, v)$ = degradation function
- S_n, S_f = noise & image power spectra
- H^* = complex conjugate of H

Wiener Filter

Noisy patch: $g = [10, 50, 100]$

Noise variance $\sigma_n^2 = 25$, original variance $\sigma_f^2 = 100$

- Wiener coefficient: $\frac{\sigma_f^2}{\sigma_f^2 + \sigma_n^2} = \frac{100}{125} = 0.8$
- Apply filter to pixel 50: $50 \times 0.8 = 40$

Resulting patch:

$$g = [10, 40, 80]$$

Smooths the noise while preserving edges better than mean filter.



The End